Classes
&
Objects

In C++

Structures in C

- Structures provide a method for packing together data of different types.
- Once the structure is defined we can create the variables of that type using declarations similar for the in-built data types.

```
struct student
{
        char name[20];
        int roll_no;
        float marks;
};
```

struct student A;

- We can declare the variables of the type student.

Limitations of Structures in C

1. We can not treat the struct data types like in-built data types. E.g.

   struct complex c1,c2,c3;

   then c3=c2+c1; is illegal.

2. No Data Hiding: Structure members can be accessed anywhere by any function i.e. these are always public members.

3. The statements for declaring variables in C is like

   struct student s1;

4. By default members of a class are private whereas members of structure are public.

But in C++ we can write like

student s1;

C++ extends the capabilities of structure by providing facility to hide data.

Specifying a Class

- Class is a way to bind the data and its associated functions together.
- It is an Abstract Data Type can be treated like any other in built data type.
- Class Specification has two parts:

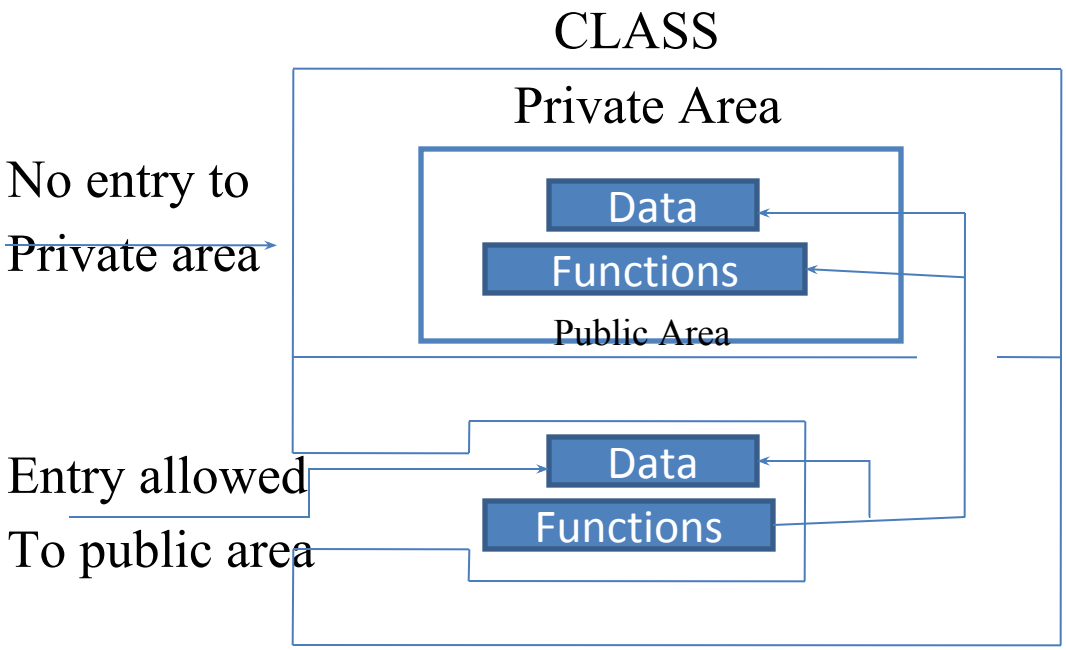
1. Class Variable Declaration
2. Class Function Definitions

```
Class class_name
  {
        private:
              variable declaration;
              function declaration;
         public:
              variable declaration;
              function declaration;
  };
```

- In C++, **class creates a new data type that may be used to create objects** of that type. Therefore, an object is an instance of a class in just the same way that some other variable is an instance of the **int data type, for example. Put differently, a class is** a logical abstraction, while an object is real. (That is, an object exists inside the memory of the computer.)

- The variables and functions in a class are collectively called *class members.*
- They are generally grouped into two sections *public & private.*
- The keywords *public & private are called visibility labels.*
- Variables declared inside the class are called *data members* and functions as member functions.
- Only the member functions can have the access to the private data members and private member functions.
- The Binding of Data and functions in a class is called ENCAPSULATION.

Data Hiding in Classes



Hence using a **CLASS** keyword we can create the new data type. This data type has the data and functions in itself.

Normally data declaration is kept in the private section and the functions are declared in the public section.

The Private data can be accessed by the member function of this class only.

Creating Class Objects

- The declaration of the objects is similar to the variables of the basic data type.
- Important: Class declaration does not allocate any memory space for the objects. The memory space to the objects is allocated at the time of declaration of objects. E.g. Item x,y,z;
- Objects can also be created at the time of class declaration. Like;

```
class item
  {
     ::::::::::::
  } x,y,z,;
```

This is practice is not followed so that the declaration of the objects is near to the place of their use.

Accessing Class Members

- The private data of a class can be accessed only through the member functions of that class.
- Following format is used to call the member function

  object-name.function-name(actual-arguments);

  e.g. x.getdata(100,76.8);
- Following statements are wrong statements:

  getedata(100,76.8);

  or x.number=100;
- Objects communicate with each others by sending and receiving messages. This is achieved through the member functions.

Defining Member Functions

- Member Functions can be defined in two places:

  1. Outside the class definition
  2. Inside the class definition

Function Definition Outside the Class definition:

The function is defined like a normal function, the only difference is that it will have membership 'identity label' in the header. This tells the compiler that to which class the function belongs to:

```
return-type class-name :: function-name (argument declaration)
{
    Function body
}
```

Differences of the normal function with the member functions

1. Different classes can have the same function name, where the membership label will resolve the scope.
2. Member Functions can access the private data of the classes, but the normal functions cannot do so.
3. A member function can call the another member function without using the dot operator.

Function Definition Inside the Class Definition:

Here in this case the function declaration is replaced by the function definition inside the class.

- Whenever the function is defined inside the class it is treated as an inline function. Hence all the restrictions applicable on the inline functions are also applicable on these functions.

Normally only the small functions are defined inside the class definition.

Example Program

```
#include<iostream.h>
class item
{
      int number;
      float cost;
public:
      void getdata(int a, float b);

     void putdata(void)
    {

     cout<<"Number:"<<number<<"\n";
        cout<<"Cost:"<<cost <<"\n";
     }
} ;
```

```
void item::getdata(int a, float b)
{
    number=a;
    cost=b;
}
 int main()
{
      item x;
  cout<<"\object x"<<"\n";

     x.getdata(100,299.95);
    x.putdata();
  item y;
 cout<<"object y"<<"\n";
y.getdata(200,175.5);
y.putdata();
return 0;
}
```

Making an outside function Inline

- It is always good practice to define the member functions outside the class definition.
- Even while defining the function outside the class we can make the function INLINE.

e.g.

```
Class item
    {
        ::::::::::
     public:
        void getdata(int a,float b);
    };
inline void item::getdata(int a, float b)
   {    number=a;
•           cost=b;
    }
```

Nesting of Member Functions

- We can call a member function of a class by the object of that class.
- Another way to access the member function is that we can call that function inside another member function, called "***Nesting of Member Functions***"

```
#include <iostream.h>
#include<conio.h>
class set
{
   int m,n;
  public:
     void input(void);
     void  display(void);
      int largest(void);
};
int set::largest(void)
{
          if(m>=n)
            return(m);
          else
                  return(n);
}
void set::input(void)
{
       cout<<"Input values of m & n"<<"\n";
        cin>>m>>n;
}
void set::display(void)
{
cout<<"Largest Value = "<<largest()<<"\n";
}
int main()
{
     set a;
    a.input();
    a.display();
    return 0;
}
```

Accessibility

Specifiers	Within Same Class	In Derived Class	Outside the Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

Private Member Functions

- There are some situations where we have to place the function in the private section. e.g deleting a customer account etc. These are the functions that can result into serious consequences.

Private member functions can be accessed only by the other member functions. Even it cannot be accessed through the object with the dot operator.

```
Class sample
{
      int m;
     Void read(void);
Public:
      void update(void);
       void write(void);
 };
```

and

```
void sample::update(void)
 {
      read();
  }
```

Now if s1 is the object of sample,

then s1.read(); is illegal.

Arrays within a class

- Arrays can be a member variable in a class.

e.g.

```
Const int size=10;
      class array
       {
          int a[size];
        public:
            void setval(void);
             void display(void);
           }
```

The member array variables can be used as a normal variables.

Static Data Members

- Static data member are class members that are declared using static keyword A static member has certain special characteristics These are:
- Only one copy of that member is created for the entire class and is shared by all the objects of that class , no matter how many objects are created.
- It is initialized to zero when the first object of its class is created .No other initialization is permitted
- It is visible only within the class, but its lifetime is the entire program.

Syntax

- static data_type data_member_name;

Example:

```
#include <iostream>
using namespace std;

class A
{
public:
        A() { cout << "A's Constructor Called " << endl; }
};
class B
{
        static A a;
public:
        B() { cout << "B's Constructor Called " << endl; }
};
int main()
{
        B b;
        return 0;
}
```

Output:

B's Constructor Called

The program calls only B's constructor, it doesn't call A's constructor. The reason for this is simple, static members are only declared in class declaration, not defined. They must be explicitly defined outside the class using scope resolution operator.
If we try to access static member 'a' without explicit definition of it, we will get compilation error

Static Member Functions

- By declaring a function member as static, you make it independent of any particular object of the class.
- A static member function can be called even if no objects of the class exist and the **static** functions are accessed using only the class name and the scope resolution operator **::**.
- A static member function can only access static data member, other static member functions and any other functions from outside the class.
- Static member functions have a class scope and they do not have access to the **this** pointer of the class. You could use a static member function to determine whether some objects of the class have been created or not.

- e.g. Class-name::function-name;

```
#include <iostream>

using namespace std;

class Demo
{
	private:
		//static data members
		static int X;
		static int Y;

	public:
	//static member function
	static void  Print()
	{
		cout <<"Value of X: " << X << endl;
		cout <<"Value of Y: " << Y << endl;
	}
};

//static data members initializations
int Demo :: X =10;
int Demo :: Y =20;


int main()
{
	Demo OB;
	//accessing class name with object name
	cout<<"Printing through object name:"<<endl;
	OB.Print();

	//accessing class name with class name
	cout<<"Printing through class name:"<<endl;
	Demo::Print();

	return 0;
}
```

Example:

Output

```
Printing through object name:
Value of X: 10
Value of Y: 20
Printing through class name:
Value of X: 10
Value of Y: 20
```

Array of Objects

- We know that class is a data type and the object is a variable of class. Hence we can have an array of variables of class type.

```
class employee
    {
        char name[30];
        float age;
     public:
         void getdata(void);
          void putdata(void);
};
```

Then array of the objects are

```
employee manager[3];
employee clerk[15];
employee worker[75];
```

Protected Access Specifier

- Protected is a visibility modifier, that serves a limited purpose in inheritance.
- A member declared as protected can be accessed by the member functions of that class and the functions in a class immediately derived from it.
- When a protected member is inherited in public mode, it becomes protected in the derived class too. i.e. it is ready for further inheritance.
- When a protected member is inherited in private mode, it becomes private in the derived class too. Also the public members also become private. i.e. it is not ready for further inheritance.
- The keywords private, public, protected may appear any no. of time in the class declaration.
- Class can also be inherited in protected mode, both the public and protected members of the base class become protected members of the derived class.

Objects as Function Arguments

- Like a variable of any other type, objects can also be used as arguments to the functions. There are two ways to do this:

1. A copy of the entire object is passed to the function. (Pass by value)
2. Only the address of the object is passed to the function.(Pass be Reference)

Passing the address is more efficient as only the address is being passed.

- An object can also be passed to the non-member functions, How ever such functions can have access to the public member functions only through the objects passed as argument to it.
- These functions cannot have access to the private member functions.

Example:

Passing an Object as argument

- To pass an object as an argument we write the object name as the argument while calling the function the same way we do it for other variables. **Syntax:**
- function_name(object_name);

Returning Object as argument

- **Syntax:**
- object = return object_name;

Const Member Functions:

- If a member function does not alter any data in the class. Then we may declare it as ***const*** member. E.g.

```
void  mul(int,int) const;
double get_bal() const;
```

It is generally appended to the function prototype, The compiler will generate an error message, If such a functions try to alter the data values.

Pointer to Members

- We can assign the address of the member of a class to a pointer.

We can get the address by using the operator &.

- A class member pointer can be declared using the operator ::* with the class name. e.g.

```
class A
   {
      private:
            int m;
       public:
           void show();
     };
```

We can define a pointer to the member as follows:

```
int A::* ip=&A :: m;
```

Here ip is a pointer to a member of class A. and address of class member of class A i.e. m is being assigned to ip.

the statement `int *ip=&m;` is wrong.

Pointer to Members

Let us assume that we have a as an object of A declared in a member function.

We can access m using ip as:

```
cout<<a.*ip;
```

Here is an another example:

```
ap=&a;
cout<<ap->*ip;
cout<<ap->m;
```

->* is called dereferencing operator is used to access a member when we use pointers to both the object and the member.

- The dereferencing operator .* is used when the object itself is used with the member pointer. Here the *ip is used like member name.

- We can also have pointers to the member functions which then can be invoked using the dereferencing operators in the main().

  (obj_name.* pointer-to-member function) ;

  (pointer-to-object->*pointer-to-member function)

Local Classes

- Classes can be defined and used in a function or a block. Such classes are called local classes. E.g.

```
void test(int a)
 {
      ::::::::::
      ::::::::::
    class student
      {
            ::::::::::::::
            ::::::::::::::
        };
         ::::::::::::::
         ::::::::::::::
         student   s1(a);
            ::::::::::::::::::
            ::::::::::::::::::
    }
```

Friendly Functions

- The private members of a class can not be accessed from outside of that class.
- In other words, a non-member function cannot have an access to the private data of a class.
- There could be a situation where we would like two classes to share a particular function.

  For example, two classes, Manager & Scientist must have the function income_tax().
- We can make income_tax() function a friend of both the Manager & Scientist classes.
- A friend function is a non-member function of a class, but it can access the private member of its friend class.
- To make an outside function 'friendly', we have to simply declare this function as a friend of the class.

Friendly Functions

```
class ABC
    {
       ……..
       ……..
     public:
       ……..
       ……..
       friend void xyz(void);
    };
```

- The friend function definition does not use either the keyword friend or the scope resolution operator :: .
- A function can be declared as a friend in any number of classes.
- A friend function has full access rights to the private members of the class.
- Friend function is not in the scope of the class to which it has been declared as friend.
- It can not be called using the object of that class.
- It can be invoked like a normal function.
- Usually it has the objects as arguments. Example:

Friend Functions in another class

Member function of one class can be a Friend function of another class. In such case they must be defined using scope resolution operator.

```
class X
{
        ……….
        ……….
        int fun1();   // member function of X
        ……….
};
class Y
{
        ……….
        friend int X :: fun1();              // fun1() of X is a friend of Y
};
```

Friend Functions in another class

- Member function of one class can be Friend function of another class. In such case they must be defined using scope resolution operator.
- All the member functions of one class can be declared as the friend functions of another class.

```
class Z
{
        ……..
        friend class X;     // All the member functions of X are
                            // friend to Y
};
```

Example:

Friend Functions in another class

Member functions of one class can be member functions of another class. In such case they must be defined using scope resolution operator.

```
class X
{
        ……….
        ……….
        int fun1();   // member function of X
        ……….
};
class Y
{
        ……….
        friend int X :: fun1();              // fun1() of X is a friend of Y
};
```

Example: